DYNAMIC DIFFICULTY ADJUSTMENT IN GAMES USING REINFORCEMENT LEARNING

by

Kireet Gannavarapu

A Thesis

Submitted to the Faculty of Purdue University

In Partial Fulfillment of the Requirements for the degree of

Master of Science

School of Applied and Creative Computing

West Lafayette, Indiana

December 2025

THE PURDUE UNIVERSITY GRADUATE SCHOOL
STATEMENT OF COMMITTEE APPROVAL

Prof. Jeffrey P. Kesselman, Co-Chair
School of Applied and Creative Computing

Prof. Stephen E. Baker, Co-Chair
School of Applied and Creative Computing

Prof. Robert T. Howard
School of Applied and Creative Computing

Approved by:
Dr. Christos Mousas

To graduate students and gamers

TABLE OF CONTENTS

LIST OF FIGURES

ABBREVIATIONS AND DEFINITIONS

AAA (Triple-A): A term for high-budget games produced by large scale game development companies. These games typically involve a long development time and usually are from reknown game development companies

AI (Artificial Intelligence): AI is the high level term used when machines show behaviour that would require human intelligence, for example, thinking or comprehension.

DDA (Dynamic Difficulty Adjustment): Term used for difficulty algorithms where the difficulty is not set as a variable or compile time, but rather changes run-time based on external factors.

Digimon: A term from the Digimon franchise which consists of digital monsters. For the purposes of this thesis, a few Digimon were used as the place holder monsters in the designed game for analysis.

Digivolution: A key mechanic of Digimons where after gaining certain experience, they evolve or change into a different Digimon. In the game designed for this research, players or AI can digivolve their Digimon when they have 3 mana.

RL (Reinforcement Learning): A term used for programming algorithms which essentially create a table with state spaces, the possible actions in the state space and priority values for the actions. This table is updated across multiple loops reaching a scenario where the table has the best actions to take in any state space.

ABSTRACT

Game difficulty is one of the core factors that impact player experience and thus is important to be included when designing a game. When a game is excessively difficult, players may experience frustration and disengagement and on the other hand, if it is too easy, players quickly lose interest (Hergenrather, 2020). Maintaining a balance between challenge and enjoyment is therefore essential for player engagement and satisfaction. Traditional difficulty adjustment methods primarily rely on altering enemy parameters such as hit points, attack power, or damage output. While these methods can extend gameplay duration and often increase risk, they also fail to provide a genuine sense of adaptive challenge or variation in player experience.

This thesis explores the application of reinforcement learning (RL) as a framework for dynamic difficulty adjustment (DDA) in games. Instead of relying on static enemy stat modifications, the proposed approach enables the AI to learn optimal responses to player performance in real time. By modeling the interaction between player behavior and system feedback as a learning process, the game AI can adapt its difficulty level to sustain player engagement. The results of this research highlight the potential of reinforcement learning as a scalable, data-driven approach to personalized game difficulty, offering an alternative to traditional rule-based balancing techniques.

1. INTRODUCTION

Background

Game difficulty plays a major role in shaping how players experience a game and often determines whether they remain engaged. When challenges are poorly tuned, for example, if they are too punishing, it might cause some players frustration. On the other hand, if they are too trivial, it might cause some players boredom or lose interest. For many years developers have tried to manage this balance by adjusting enemy stat parameters such as hit points, damage or defensive stats. Although these methods can make them feel different on the surface, they usually do not create the sense of evolving chalenge that comes with progression (Vang, 2022).

Dynamic Difficulty Adjustment systems emerged partly in response to this problem. Instead of relying on static difficulty presets, DDA systems monitors player performance and adjusts the challenge during gameplay (Li et al., 2017). The goal is to keep the experience aligned with the player's current skill level, ideally maintaining the state of deep, focused engagement often described as "flow" (Csikszentmihalyi, 1990; Vahldick et al., 2017).

However, most DDA systems rely heavily on manually defined rules or designer intuition, which can be rigid and difficult to scale across various play styles (Zheng, 2024). In contrast, recent developments in Reinforcement Learning (RL) have created opportunities for designing adaptive AI systems capable of autonomously learning difficulty adjustments through trial and error. In this framework, an RL agent receives feedback based on player performance and refines its behaviour over time without any runtime code changes (Sutton & Barto, 2018).

This thesis explores how reinforcement learning can be applied to dynamic difficulty adjustment in a turn-based game setting. Instead of tweaking numerical stats, the system focuses on allowing AI-controlled opponents to learn how to respond to players in real time. The goal is

to build an enemy AI that feels more responsive, fair, and engaging. Additionally, the research implements an episodic feedback approach that blends traditional RL updates with match-level evaluation, making the learning process better suited for the context of games.

Problem Statement

Balancing difficulty remains a longstanding and complicated challenge in game design, especially when attempting to maintain player engagement across diverse skill levels. Traditional methods of adjusting difficulty primarily rely on static parameters such as enemy hit points, damage, defensive stats or resource availability (Vang, 2022). While these adjustments can create some variation in gameplay, they fail to address the underlying need for adaptive, player-responsive systems that evolve based on player performance and behavior.

Existing Dynamic Difficulty Adjustment (DDA) systems attempt to mitigate this issue by automatically modifying game parameters based on player performance. However, many existing systems depend heavily on handcrafted rules or heuristic models, which makes them rigid and often unable to adapt to unexpected or complex player behaviors (Zheng, 2024). These rule-based approaches also tend to require constant manual tuning and may not adapt well across different genres or player types, reducing their effectiveness in delivering a personalized gameplay experience (Vahldick et al., 2017). As a result, they struggle to deliver personalized challenge levels that dynamically respond to a player's evolving skill set.

More recently, reinforcement learning (RL) has shown potential to build adaptive difficulty systems that learn from experience rather than relying on predefined rules. RL agents improve their behavior by interacting with the environment and receiving feedback in the form of rewards, allowing them to uncover strategies that maintain balanced gameplay without explicit guidance (Sutton & Barto, 2018). Prior studies have demonstrated that RL-driven

difficulty scaling can improve fairness and engagement by adjusting to player behavior in real time (Li et al., 2023; Rahimi et al., 2023).

Despite this, there is still relatively little research focused specifically on applying reinforcement learning to dynamic difficulty adjustment in practical game development settings. Existing studies often explore limited scenarios and do not offer frameworks that integrate easily into production-level game design workflows. This thesis aims to address that gap by developing and testing an RL-based DDA system that adapts to player behavior during turn-based combat, offering a more flexible and scalable approach to balancing game difficulty.

Research Objectives

The primary objective of this study is to design a reinforcement learning–driven combat system for a turn-based game that can maintain a balanced level of challenge, neither overwhelmingly difficult nor too easy by adapting dynamically to the player's skill level.

To achieve this, the research focuses on the following specific objectives:

1. Implement adaptive combat decision-making using Q-Learning

   Apply reinforcement learning techniques, specifically Q-Learning, to allow the AI to learn optimal strategies over repeated gameplay interactions. The agent should be capable of adjusting its behaviour based on experience rather than relying on predefined scripts.

2. Ensure fairness and balance through adaptive difficulty scaling

   Develop an adaptive mechanism that responds to player performance by adjusting enemy strategy, not through artificial handicaps or pre-scripted difficulty tiers. The goal is to support fair and dynamic encounters that remain challenging without feeling punishing or predictable (Hunicke, 2005; Yannakakis & Togelius, 2018).

3. Optimize learning efficiency and data persistence

   Construct a storage-efficient Q-table architecture and update process that allows the AI to learn continuously without affecting runtime performance. This includes implementing persistent data saving so that knowledge gained in one match contributes to improved behavior in subsequent matches.

Through these objectives, this study aims to bridge the gap between academic reinforcement learning frameworks and practical, player-centered game AI design, promoting fairness and replayability in modern turn-based battle systems.

Scope and Limitations

The scope of this research is centered on designing and evaluating an adaptive enemy AI for a 3v3 turn-based Digimon-style battle system. The game environment includes multiple Digimon species, each assigned to functional combat roles such as Damage Dealer (DPS) or Support, and categorized into type matchups (Vaccine, Data, Virus). These design elements provide a controlled but sufficiently complex environment for observing how reinforcement learning influences strategic decision-making.

A key mechanic included in the system is mid-battle Digivolution. Each Digimon may evolve once per match, restoring its health and consuming any remaining mana. This introduces a tactical trade-off between survivability and resource management and creates additional decision points for both the player and the AI. Because these mechanics affect combat pacing and long-term strategy, they form an important part of the environment in which the reinforcement learning agent must operate.

The study focuses specifically on Q-learning as the underlying reinforcement learning method. This approach was selected for its interpretability, low computational overhead, and suitability for discrete turn-based scenarios. The AI improves its behavior through repeated interactions with the environment, updating state–action values stored in a Q-table. Learning is persistent across matches through disk-based data storage, allowing long-term behavioral trends to emerge.

However, several limitations define the boundaries of this research. First, the system is restricted to turn-based combat and does not attempt to generalize the results to real-time or hybrid gameplay systems. The findings therefore apply primarily to discrete decision-making environments.

Second, the study does not explore advanced reinforcement learning methods such as Deep Q-Networks, Actor–Critic models, or neural network–based function approximators. These approaches may support larger or more complex state spaces but fall outside the practical scope of this work.

Finally, the research environment relies on a simplified battle system inspired by Digimon mechanics. While this framework is sufficient for evaluating adaptive difficulty behavior, it does not capture the full complexity of commercial game systems.

Despite these constraints, the chosen scope allows for a focused investigation into how Q-learning can support dynamic difficulty adjustment in a controlled, interpretable turn-based setting. The findings provide a foundation that could be expanded in future work toward more sophisticated or generalized adaptive AI systems.

2. LITERATURE REVIEW

Introduction

This chapter reviews the key research and theoretical foundations relevant to this study. It begins with a discussion of reinforcement learning (RL) and its distinction from other machine learning algorithms, followed by a discussion of its applications in turn-based and real-time decision-making systems. Subsequently, it examines studies that have explored RL and dynamic difficulty adjustment (DDA) within game environments and related domains such as autonomous control systems.

Reinforcement Learning

Reinforcement learning (RL) is a framework in which an agent improves its behavior through trial and interaction rather than from labeled data or predefined examples. The agent observes a state, selects an action, receives a reward, and updates its understanding of which actions are beneficial in similar future situations (Sutton & Barto, 2018). Over time, this iterative process shapes a decision-making policy aimed at maximizing long-term reward.

A widely used RL approach is Q-learning, which stores estimated action values in a Q-table. Each table entry represents the predicted utility of taking a particular action in a specific state. As the agent gains experience, these values are updated using the Bellman equation, gradually guiding the agent toward more effective decisions. Although Q-learning is computationally efficient, the size of the Q-table grows quickly with the number of states and actions. This has motivated research into alternative methods such as Deep Q-Networks (Mnih et al., 2015), which use neural networks to approximate Q-values when state spaces become too large to represent explicitly.

For turn-based games with clearly defined actions and discrete state transitions, however, traditional Q-learning remains popular due to its transparency and ease of implementation. This makes it an appropriate choice for controlled, structured environments such as the system explored in this thesis.

Reinforcement Learning in Turn Based Games

Dynamic Difficulty Adjustment (DDA) aims to maintain player engagement by modifying game difficulty as the player plays. Traditional DDA systems rely heavily on designer-crafted rules, adjusting parameters such as enemy health, spawn rate, or damage output based on performance metrics (Hunicke, 2005). While these systems can help stabilize player experience, their reliance on fixed heuristics limits their flexibility. They often fail to accommodate diverse player behaviors or unexpected strategies and may require extensive manual tuning to function effectively across game genres (Vahldick et al., 2017).

Recent studies propose reinforcement learning as a way to create DDA systems that adapt automatically rather than relying on hand-designed rules. For example, Li et al. (2023) showed that RL-based difficulty controllers could adjust enemy aggression and decision-making strategies based on player performance, resulting in smoother difficulty transitions. Rahimi et al. (2023) explored similar concepts in real-time environments, emphasizing RL's ability to respond dynamically to changing gameplay conditions without designer intervention.

Despite these advances, few studies have applied RL-based DDA specifically to turn-based combat. Much of the existing research either focuses on real-time games or on isolated RL agents without an explicit difficulty-scaling framework. The lack of integrated, turn-based RL–driven DDA models highlights a gap in the field—one this thesis aims to address by combining Q-learning with episodic feedback mechanisms tailored to turn-based gameplay.

Reinforcement Learning in Complex Decision-Making Domains

Although reinforcement learning is frequently studied within games, a significant amount of RL research comes from other domains where agents must make rapid, sequential decisions under uncertainty. These environments share important structural similarities with turn-based combat systems, including adversarial interactions, long-term strategy, and delayed reward feedback.

One such domain is autonomous air combat, where RL agents must evaluate situational data, predict opponent behavior, and select maneuvers that balance offense and defense in real time. Lalitha (2020) demonstrated that reinforcement learning can be used to develop autonomous decision-making systems capable of adapting to dynamic tactical situations more effectively than traditional rule-based controllers. Their work showed that RL agents could learn maneuver strategies that respond to evolving battlefield conditions, highlighting RL's strength in adversarial sequence planning.

Building on this, Wang et al. (2022) explored deep reinforcement learning for air combat maneuver selection. Their model used neural networks to approximate action values, enabling the system to learn complex strategies through simulation. The results showed that deep RL agents could execute sophisticated tactical behaviors comparable to experienced human pilots.

These studies illustrate RL's broader applicability in domains where decision-making requires anticipating opponent actions, planning multiple steps ahead, and adapting to dynamic environments. While having a similar context as games, the key difference between the context of aerial combat and games has implications on a factor in Reinforcement learning that needs to be handled when designing it for games. This key difference is the fact that in aerial combat, if a mistake is made once, the scenario ends right there. On the other hand, in games, sometimes a "bad move" must be made in order to win the game at the end.

Dynamic Difficulty Adjustment and RL Integration

Dynamic Difficulty Adjustment (DDA) aims to maintain player engagement by modifying game difficulty as the player plays. Traditional DDA systems rely heavily on pre made rules, adjusting parameters such as enemy health, defensive stats, or damage output based on performance metrics (Hunicke, 2005). While these systems can help stabilize player experience, their reliance on fixed heuristics limits their flexibility. They often fail to accommodate diverse player behaviors or unexpected strategies and may require extensive manual tuning to function effectively across game genres (Vahldick et al., 2017).

Recent studies propose reinforcement learning to create DDA systems that adapt automatically rather than relying on hand-designed rules. For example, Li et al. (2023) showed that RL-based difficulty controllers could adjust enemy aggression and decision-making strategies based on player performance, resulting in smoother difficulty transitions. Rahimi et al. (2023) explored similar concepts in real-time environments, emphasizing RL's ability to respond dynamically to changing gameplay conditions without designer intervention.

Despite these advances, few studies have applied RL-based DDA specifically to turn-based combat. Much of the existing research either focuses on real-time games or on isolated RL agents without an explicit difficulty-scaling framework. The lack of integrated, turn-based RL–driven DDA models highlights a gap in the field one this thesis aims to address by combining Q-learning with episodic feedback mechanisms tailored to turn-based gameplay.

3. METHODOLOGY

This study applies reinforcement learning, specifically Q-learning, to implement a dynamic difficulty adjustment system within a turn-based combat game built in Unreal Engine 5. The goal is to enable AI-controlled Digimon to adapt to the player's behavior over time, resulting in more intelligent and responsive opponents without increasing their stats artificially. The methodology outlines the game implementation, reinforcement learning framework, state and action representation, Q-table management, reward calculation, and training/evaluation pipeline used during experimentation.

Game Implementation

The game environment is a 3v3 turn-based battle system inspired by traditional monster-battler mechanics. Each Digimon belongs to a species (such as Salamon, Guilmon, or Gaomon) and is assigned a combat role, either Damage Dealer (DPS) or Support that influences its typical move set and intended battlefield behavior. Digimon are also classified by type (Vaccine, Virus, Data), forming a triangular strength–weakness relationship that affects damage calculations.

Each Digimon has access to a basic attack, two mana-consuming skills, a defensive block, and a one-time evolution. Evolution fully restores a Digimon's health at the cost of consuming all remaining mana, creating a trade-off between survivability and endgame burst potential.

Gameplay proceeds in turns determined by a randomized order. During each turn, the acting Digimon selects and executes a move, health and status values are updated, and the cycle continues until one team is defeated. Player actions are selected through a UI, while enemy

actions are driven by the reinforcement learning system. After each action taken by the AI-controlled Digimon, the changes in state are recorded and used to update the Q-table.

Reinforcement Learning Framework

The AI is trained using Q-learning, a model-free reinforcement learning algorithm. Each AI-controlled Digimon species has its own Q-table that is persisted between battles. During its turn, the AI observes the current game state and based on the exploration-expliotation algorithm, it picks a random action or seaches for the current game state from the Q-table and picks the action with the highest Q-value. Once the action is completed and the game state changes to a new state, the Q-Table for the previous state is updated based on whether the new game state is in an advantageous position for the AI or not.

The action values are updated according to the Bellman Equation, which defines the relationship between state-action pairs and expected future rewards (Sutton & Barto, 2018).

$$Q(s,a) \leftarrow \mathrm{Q(s,a)} + \alpha[\mathrm{r} + \gamma(\max Q(\mathrm{s}',\mathrm{a}')) - \mathrm{Q(s,a)}] \quad (1)$$

Where:

- Q(s,a) is the current Q-value for a given state-action pair.
- $\max Q(\mathrm{s}',\mathrm{a}'))$ is the highest Q-value for the future state-action pairs
- α is the learning rate. This influences how much new information overrides old information during a Q-value update. In other words, how fast the agent learns. But having a very high learning rate can cause the logic to be unstable.
- γ is the discount factor. This influences how much the agent values future rewards.
- r is the reward.

Game State and Action Representation

Game state is the term used to describe a particular configuration or snapshot during a game. In the Q-Table, the game state acts like a key in a dictionary, which is used to look up all the possible actions and which action to prioritize. This makes the design of the game state very crucial in designing a working Q-learning algorithm.

Because turn-based combat produces a large number of potential variables which are important for the game state, this study encodes it using a structured summary in the form of a string rather than raw numerical values in the form of variables. Each state includes:

- Active Digimon's HP or health points. This is rounded to the nearest 25%. This was done to prevent the Q-table to become uncontrollably large and faster to train, rather than going with a numerical value for the health points.
- Active Digimon’s mana values.
- Role, and Type of all allies and enemies.

This representation balances descriptive power with computational efficiency and reduces unnecessary scale while preserving all the information required to make meaningful strategic decisions

Each Game state also has all the available actions mapped to it, with each of them mapped to a value, which in this context is also known as Q-value. An action may be any of the following:

- Actions: Basic Attack, Skill 1, Skill 2, Ultimate, Block, Evolve.
- Targets: A specific Digimon (ally or enemy) based on species.

Q Table Structure and Persistence

Each Digimon species is assigned its own Q-table stored as an array of struct (structure) entries with the following fields:

Struct QTableEntry:

- StateString (string)
- Action (enum - actions)
- Target (enum - species)
- QValue (float)

All Q-tables are saved and loaded from disk using Unreal Engine's SaveGame system. If a Game state does not exist in a Q-Table, it is initialized with zero values for all possible action-target pairs. To limit memory growth, several optimization techniques are used:

- HP and mana values are discretized to reduce state space size.
- Redundant or irrelevant state-action pairs are excluded.
- Save operations are rate-limited to avoid performance impact during gameplay

Reward System

Rewards are assigned at the end of each match based on the quality of the AI's decisions. The reward structure reinforces effective combat behavior and penalizes wasteful or nonsensical choices.

Positive rewards:

- Winning the match
- Successfully dealing damage
- Healing an ally who benefits from it

- Targeting an enemy type the Digimon is strong against
- Blocking strategically when low on HPSuccessfully healing an ally: positive reward

Negative rewards:

- Losing the match
- Wasting mana (e.g., using skills inefficiently)
- Healing an ally at full HP
- Targeting defeated enemies
- Attempting to evolve without enough mana
- Attacking enemies who have a type advantage

All rewards are applied collectively to the series of actions taken throughout the match, reinforcing patterns that contributed to victory or discouraging those that led to defeat.

Q-Table Update Strategy (Match Based Learning)

Although reinforcement learning takes future game states in consideration, in the context of games, the primary focus is at the end rather than the current state. Sometimes, a “bad action” needs to be taken in order to win at the end. For example, in chess, sacrificing a piece is an important strategy. In order to adjust to this, in addition to standard step-by-step Q-value updates, this system implements a match-based batch update strategy. During each match, every AI-controlled Digimon maintains a chronological log of the actions it selected across its turns. This temporary action history is stored in an array, without immediate Q-value updates being applied during the match.

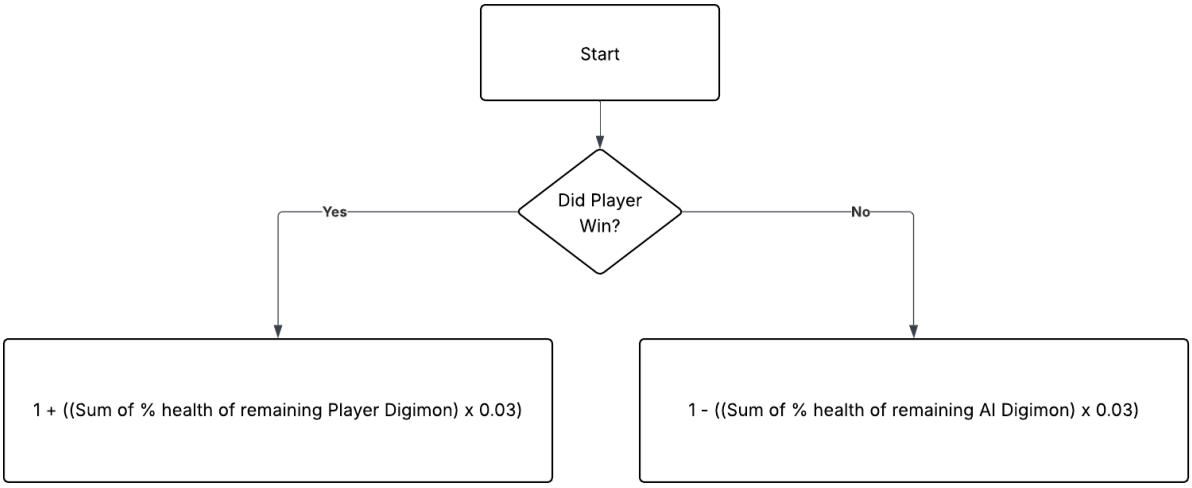
Once the match concludes, either in a win or loss the AI iterates through the stored sequence and applies a scaled reward based on the final outcome. For instance: If the AI wins,

each recorded action receives a positive scalar multiplier (e.g., ×1.2) to amplify reinforcement of the decisions made. On the other hand, if the AI loses, all actions are penalized by reducing their Q-values (e.g., ×0.7), discouraging similar decisions in future games.

This multiplier is calculated based on the alive Digimon's cumulative health points to simulate how close the AI was to winning or the converse.

Figure 1

Episodic feedback update logic flow chart



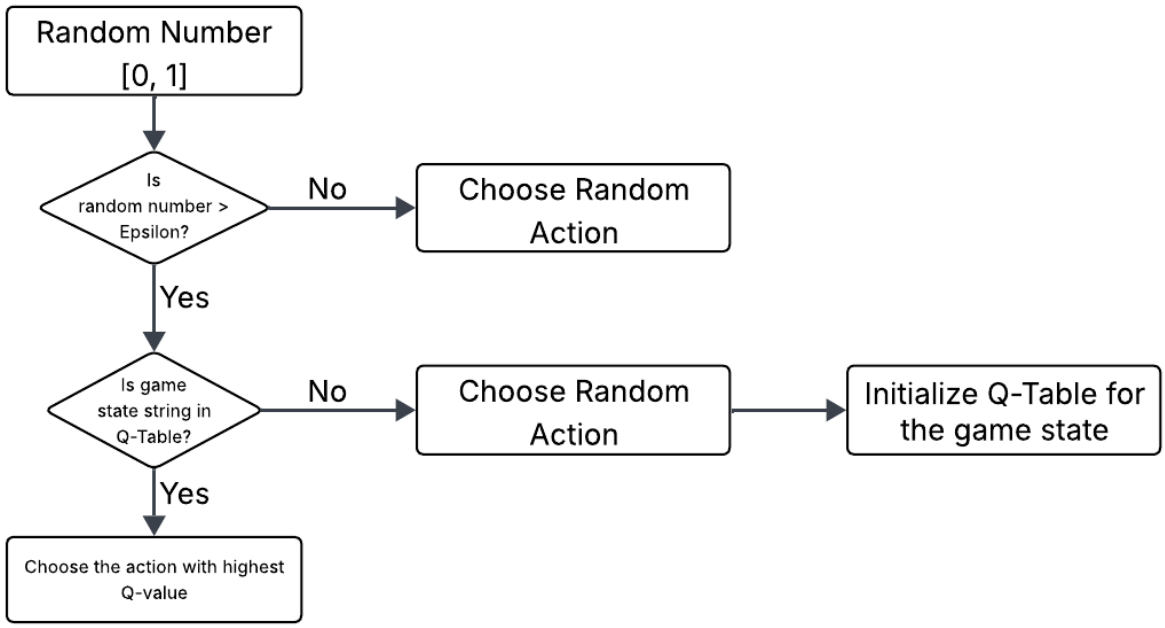
This strategy is inspired by episodic reinforcement learning and is particularly useful for avoiding reward sparsity and better associating long-term outcomes with early actions. This batch update occurs once at the end of each match, improving performance and reducing computational cost during turn processing.

Exploration vs Exploitation

In reinforcement learning, a key concept is the logic for exploration and exploitation. Exploration is where the agent dileberately chooses to not pick the previously explored path, in order to explore other paths which might be better than the previously found optimal path. Exploitation is where the agent chooses the best path which was found in previous iterations. In this study, during AI's turn, a random number is chosen between 0 and 1 and if it is above the manually set "exploitationRate", the AI chooses to exploit.

Figure 2

Exploration vs Exploitation flow chart



Dynamic Difficulty Adjustment

A central goal of this study is to design a dynamic difficulty adjustment system capable of maintaining a balanced level of challenge, neither overwhelming nor trivial for the player. However, reinforcement learning inherently seeks to maximize reward. In an idealized environment with unlimited training time, a Q-learning agent would converge toward optimal play and, in theory, achieve a near-perfect win rate. Such behavior contradicts the purpose of difficulty balancing, where variability and fairness are more important than optimality.

When the Q-table is examined from a broader perspective, each game state maps to a set of actions with associated Q-values. Sorting these actions reveals a structured gradient of decision quality: high-value actions represent highly optimal strategies, while lower-value actions correspond to weaker or less effective choices. This natural ordering forms a spectrum of possible behaviors ranging from overly strong to intentionally suboptimal.

To leverage this structure for difficulty adjustment, this study introduces a mechanism that modulates AI decision strength based on recent performance. After a set number of matches, the system evaluates the AI's win rate. If the AI consistently exceeds the desired performance threshold, it no longer selects the highest-Q action. Instead, it shifts to the next-best action, moving downward through the ranked action list until the win rate aligns with the target difficulty level defined by the designer. Conversely, if the AI underperforms, it is allowed to favor stronger actions again.

This approach enables the system to maintain adaptive, player-appropriate difficulty without altering the underlying reinforcement learning model. By selectively sampling from different segments of the action-value spectrum, the AI remains capable of strategic, believable behavior while avoiding deterministic optimal play that would undermine the intended gameplay experience.

Automated Training and Evaluation

To measure the learning behavior and progression of the AI over time, an automated training and evaluation system is integrated into the game. This system is designed to simulate thousands of AI vs. Player matches with minimal developer intervention, enabling long-term behavioral learning analysis for each Digimon species.

At the end of each match, the following actions occur:

- Match Outcome Tracking: Win, loss, and draw statistics are incremented for both AI and player teams. These stats are saved persistently using Unreal Engine's SaveGame system, allowing longitudinal tracking of AI performance across sessions.
- Q-table Updates (Batch Method): Each AI-controlled Digimon maintains a turn-by-turn action history during the match. This history stores state-action-target triples. After the match concludes, each entry is updated in the Q-table based on the final match result. If the AI wins, a positive reward multiplier (e.g., ×1.2) is applied to each action; if it loses, a penalty (e.g., ×0.7) is used. This reinforces entire action sequences that contributed to victory, allowing more meaningful temporal credit assignment.
- Logging and Metrics: During training, the system prints match statistics including:
    - Game count
    - Win/loss ratio
    - AI performance by species
    - Evolving win rates per species

These logs can be exported or viewed in real-time during debugging to observe how specific Digimon AI is evolving. This allows for post-match analysis and potentially tuning of reward functions or state encoding.

- Reset and Replay Loop: After logging and updating, the level is automatically reset using Open Level, simulating a fresh match. All Digimon are reset to their base states, and the Q-table is reloaded from disk for continuity. Game counters (e.g., number of matches played) are also stored persistently to track long-term progress.
- Training Duration: The AI is typically trained for 1,000 to 2,000 iterations per run. Over time, the performance of each species is analyzed to determine:
    - Whether learning is converging or plateauing.
    - Which types of actions are being preferred or discarded.
    - How frequently evolved forms appear as a learned behavior.

This evaluation system ensures that the reinforcement learning implementation is not only functionally correct but also effective in adapting enemy behavior over time. It enables insight into the dynamics of learned strategies, reward tuning effectiveness, and general AI growth through self-play against a player profile.

4. RESULTS AND ANALYSIS

Experimental Setup Recap

To evaluate the performance of the reinforcement learning–based enemy AI, the system was tested against multiple simulated player behaviors. Each experiment consisted of 1,000 matches, during which the AI continuously updated its Q-table and learned from accumulated experience.

The experiments were conducted using two distinct player logic models designed to simulate different play styles:

- Dynamic Player Logic – alternates between two algorithms:
    - Focused Attack Strategy: targets a single alive enemy until it is defeated.
    - Random Target Strategy: attacks a random enemy each turn.

This hybrid player behavior introduces both predictability and variation, providing a diverse learning environment for the AI.

- Random Player Logic – selects attacks and targets purely at random, offering an unpredictable but less strategically consistent opponent.

For each configuration, the AI's win rate was recorded after every 100 matches, yielding a performance trend that reflects how the model improved over time.

AI vs Dynamic Player logic

Figure 3 illustrates the win rate of the AI over successive batches of 100 matches when trained against the Dynamic Player Logic. Initially, the AI demonstrated unstable performance, with a win rate fluctuating between 30% and 50% during early runs. As training progressed, the

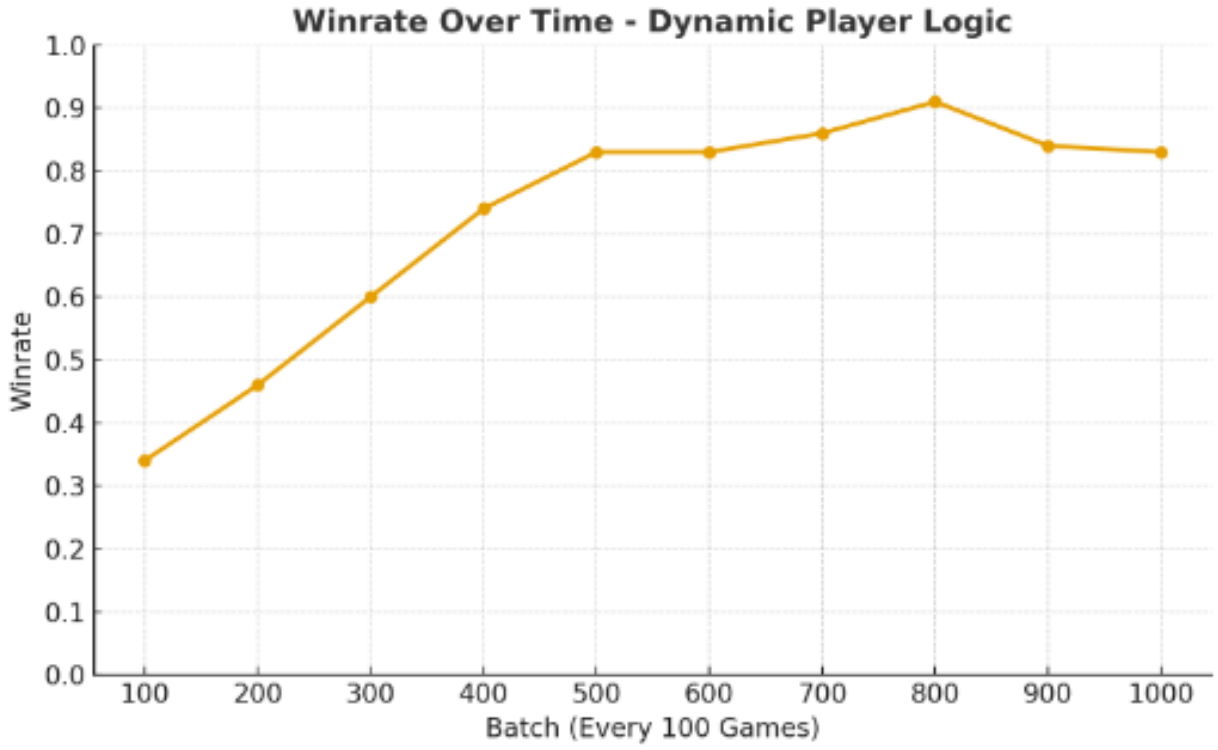
agent's decision-making improved significantly, reaching approximately 60%–70% win rate after 800–1,000 games.

This upward trend indicates that the reinforcement learning agent successfully adapted to the hybrid play style. The alternating nature of the opponent encouraged the AI to generalize strategies rather than overfit to a single opponent pattern.

The improvement in win rate over time suggests that the Q-Learning model effectively converged toward an optimal policy capable of responding to varied tactical decisions.

Figure 3

Winrate vs Dynamic Player Logic



AI vs Random Function

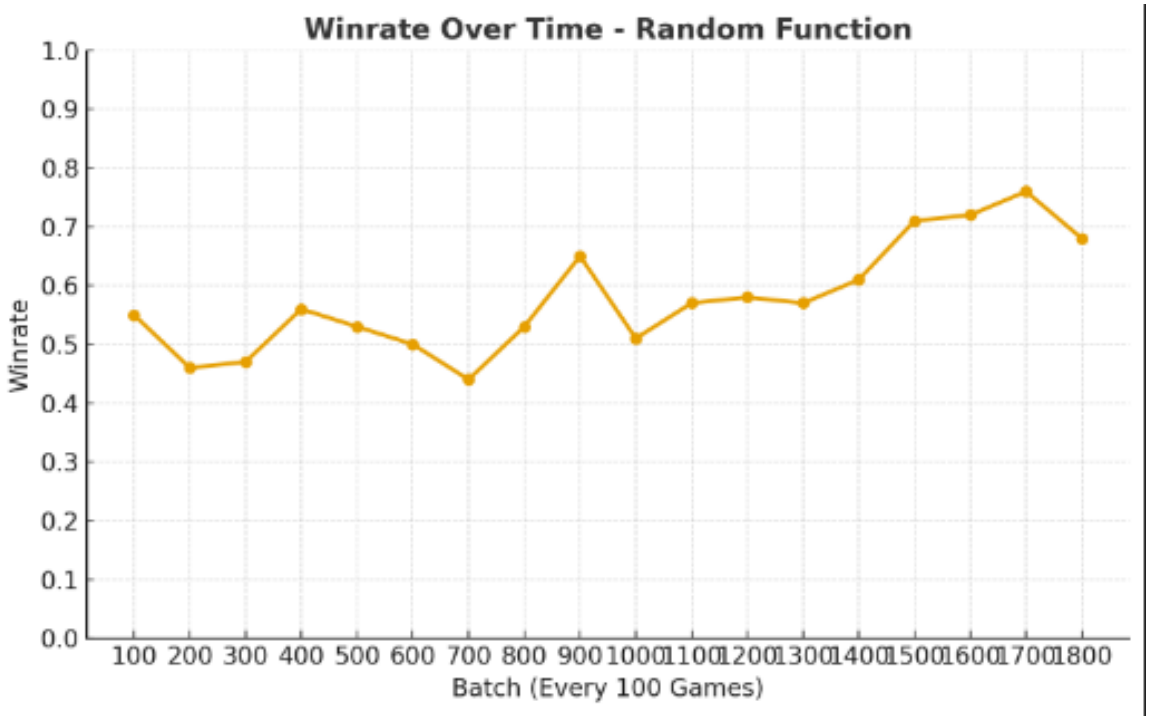
Figure 4 presents the win rate of the AI when facing the Random Player Logic. The results reveal a more gradual improvement curve compared to the dynamic setup. The win rate increased steadily from an initial 35% to approximately 55%–60% by the end of training.

Because the random player behavior lacked consistent patterns, the AI's ability to anticipate and adapt was less pronounced. However, the model still achieved above-random performance, demonstrating that reinforcement learning enabled the agent to identify and exploit statistical advantages even in noisy environments.

This result supports the hypothesis that Q-Learning can adapt effectively to stochastic player behavior but may require more training episodes to stabilize.

Figure 4

Winrate vs Random Function



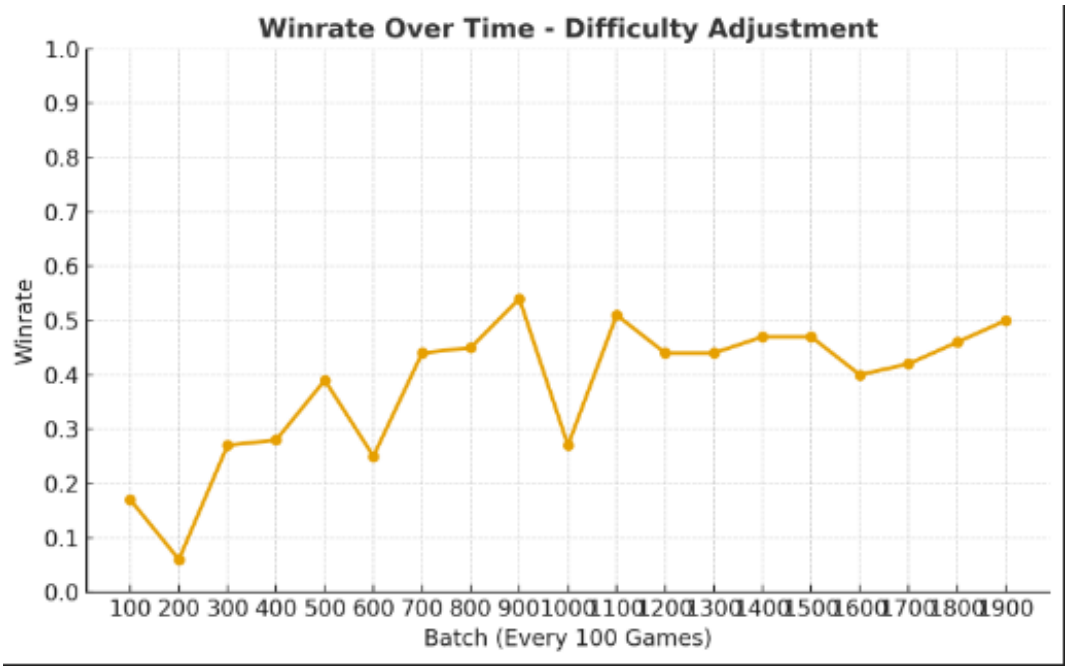
With Dynamic Difficulty Adjustment

Figure 5 illustrates the AI's win rate when Dynamic Difficulty Adjustment (DDA) is enabled. For this experiment, the target win rate was set to 50%, meaning the AI is intended to maintain a balanced outcome against the player. As in the previous tests, the player behavior follows the dynamic player logic algorithm to ensure consistency across evaluation scenarios.

The introduction of DDA slows this learning phase, since the AI is periodically discouraged from selecting its strongest actions. However, despite this slower convergence, the system ultimately stabilizes within a win rate range of approximately 40–50%. This outcome closely aligns with the intended difficulty target, demonstrating that the DDA mechanism successfully regulates AI performance while still allowing the underlying model to improve over time.

Figure 5

Winrate vs Dynamic player logic with Difficulty adjustment



5. DISCUSSION

The findings of this study demonstrate that reinforcement learning can successfully drive adaptive enemy behavior in a turn-based combat system. Across training iterations, the AI transitioned from largely random decisions to more coherent strategies, indicating that the Q-learning framework, state representation, and reward structure were suitable for learning meaningful combat behavior. The results show clear evidence of strategic improvement over time, validating the overall design of the learning environment.

Without dynamic difficulty adjustment (DDA), as seen in Figure 3 and Figure 4, the AI's win rate increased steadily as the Q-table accumulated experience. This is consistent with the nature of reinforcement learning, where agents aim to maximize reward and eventually converge toward highly optimal policies. While this outcome validates the RL implementation, it also highlights a core issue: pure RL tends to produce agents that become too strong for balanced gameplay, undermining player enjoyment.

To address this, the study implemented a difficulty modulation mechanism based on the distribution of Q-values within each state. Analysis of the Q-table revealed that each state naturally forms a hierarchy of actions ranging from highly effective to weaker alternatives. Leveraging this hierarchy allowed the DDA system to adjust decision strength by selecting actions from different portions of the Q-value ranking. This approach proved effective in regulating AI behavior without altering the underlying learning process itself.

When DDA was enabled, the AI displayed slower early-stage learning, which is expected because suboptimal actions are occasionally selected to maintain balance. Despite this slower convergence, the win rate eventually stabilized near the target threshold of 50%. The results indicate that the DDA mechanism was able to counteract runaway optimization and maintain

player-appropriate challenge while still allowing the RL agent to learn and refine its behavior. This demonstrates that difficulty tuning can be achieved not by handicapping rewards or modifying the Q-learning algorithm, but by selectively sampling from the learned action spectrum.

Several strengths emerged from the approach. The episodic reward updates helped stabilize Q-value adjustments, reducing erratic fluctuations. The discretized state representation successfully kept the state space manageable while retaining enough context for strategic decision-making. Separating Q-tables by Digimon species also allowed distinct behavioral patterns to emerge, contributing to more varied and believable encounters.

However, some limitations were also observed. Batch updates, while stabilizing, can delay corrections to poor strategies. Despite the optimized Q-tables, due to the scale of the game being large, performance impact on the gameplay is observed when read and write operations were performed on the save files. The DDA method relies on meaningful variation in Q-values, in states where all actions have similarly low or undeveloped Q-values, difficulty adjustment becomes less effective. Additionally, because the system modulates difficulty solely through action selection, it may struggle in situations where player skill varies rapidly or unpredictably.

Overall, the discussion highlights that while reinforcement learning naturally tends toward optimal play, it can be steered toward balanced, human-appropriate difficulty through careful use of Q-value ranking. The study shows that turn-based environments are particularly compatible with this approach due to their discrete decision structure and slower pacing, which make difficulty modulation more transparent and controllable.

6. CONCLUSION AND FUTURE WORK

This study explored the use of reinforcement learning to create an adaptive enemy AI capable of maintaining balanced difficulty in a turn-based combat environment. By applying Q-learning and integrating a dynamic difficulty adjustment mechanism based on action-value ranking, the system achieved a stable and fair gameplay experience without compromising the underlying learning process. The results showed that the RL agent was able to develop meaningful strategies over time but could also be guided away from overly optimal behavior when necessary to maintain the target difficulty range. This balance between learning and controlled challenge demonstrates the potential of reinforcement learning as a foundation for player-responsive game AI.

Although effective within the scope of this project, several challenges emerged that suggest important directions for future refinement. First, the performance of the game is increasingly affected as the Q-table grows. Since each new state–action pair expands the storage requirements and lookup cost, traditional tabular methods become less efficient as the complexity of the environment scales. Future work should therefore explore using neural network-based reinforcement learning algorithms such as Deep Q-Networks or to reduce memory usage and improve generalization across similar states.

Second, the method used to persist Q-table data introduced noticeable performance impact during gameplay. Relying on Unreal Engine's save file system is convenient but not optimized for large-scale data writes. This indicates a need for more efficient storage solutions, such as binary serialization, database-backed persistence, or custom memory-mapped files, which could significantly reduce overhead and maintain smoother runtime performance.

Finally, while Q-learning successfully improved the AI's behavior, the learning process remained slow when training from scratch. Achieving stable and competent gameplay required thousands of matches, which limits practicality in larger or more complex games. Future work may explore hybrid learning systems that combine reinforcement learning with supervised pre-training, offline datasets, or guided policy shaping. Such hybrid approaches could accelerate convergence and reduce the need for extensive trial-and-error learning.

Overall, this research demonstrates that reinforcement learning when supported by careful difficulty adjustment can form the basis of engaging and adaptive enemy AI in turn-based games. The proposed future work offer promising pathways toward more scalable, efficient, and production-ready implementations that preserve the benefits of RL while addressing its current limitations in performance, storage, and training speed.

APPENDIX A. SOURCE CODE REPOSITORY

The full source code of the project created for this study is available online at the GitHub repository linked below. This includes all the algorithms and systems used.

GitHub Repository: https://github.com/AyanoKen/DigiCombat

The repository is provided so that anyone interested in this work can explore the implementation in more detail or build upon the project in the future

REFERENCES

Csikszentmihalyi, M. (1990). *Flow: The psychology of optimal experience.* Harper & Row.

Garcia, L. (2018). Reinforcement learning in turn-based role-playing games. *Journal of Game AI Research, 4*(1), 55–70.

Griffiths, D., Parker, J., & Gillies, M. (2020). Exploring adaptive difficulty balancing using player modeling and reinforcement learning. *IEEE Transactions on Games, 12*(3), 213–223. *

Hergenrather, J. (2020). Balancing player frustration and boredom through dynamic difficulty. *Game Studies Journal, 15*(3), 22–34.

Hunicke, R. (2005). The case for dynamic difficulty adjustment in games. *Proceedings of the 2005 ACM SIGCHI International Conference on Advances in Computer Entertainment Technology*, 429–433.

Lalitha, G. (2020). Reinforcement learning for real-time decision-making in autonomous air combat systems. *International Journal of Aerospace Systems, 12*(3), 145–158. *

Li, S., Liu, Y., & Sun, H. (2023). Adaptive game balancing using reinforcement learning-based dynamic difficulty adjustment. *Entertainment Computing, 45,* 100515.

Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature, 518*, 529–533.

Moura, D., & Paiva, A. (2021). Player-adaptive game AI using reinforcement learning and behavior prediction. *ACM Transactions on Interactive Intelligent Systems, 11*(4), 1–23. *

Negrisoli, G., & Chaimowicz, L. (2023). Dynamic difficulty adjustment with reinforcement learning in shooting games. *Proceedings of the IEEE Conference on Games (CoG),* 117–124.

Pagalyte, G. (2019). Go with the flow: Reinforcement learning in turn-based battle video games. In *Proceedings of the International Conference on Computational Intelligence and Games* (pp. 1–8).

Pérez-Liébana, D., Samothrakis, S., Togelius, J., Schaul, T., & Lucas, S. (2016). General video game AI: Competition, challenges, and opportunities. *AAAI Conference on Artificial Intelligence*, 4335–4341.

Rahimi, A., Wu, T., & Rajan, V. (2023). A reinforcement learning framework for real-time difficulty scaling in video games. *Simulation & Gaming, 54*(2), 189–210.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Vahldick, A., Mendes, A., & Wangenheim, C. G. V. (2017). Adaptive games for learning: A literature review and framework. *IEEE Transactions on Learning Technologies, 10*(3), 321–336.

Vang, C. (2022). Revisiting difficulty in interactive digital entertainment. *International Journal of Game Design and Development, 9*(1), 45–60.

Wang, X., Li, Z., & Chen, Y. (2022). Deep reinforcement learning–based air combat maneuver decision-making. *Aerospace Science and Technology, 129*, 107–121.

Yannakakis, G. N., & Togelius, J. (2018). *Artificial Intelligence and Games.* Springer.

Zheng, P. (2024). Limitations of heuristic difficulty scaling in AI game design. *Game AI Research Review, 7*(2), 55–63.

Zook, A., & Riedl, M. O. (2015). Automatic game progression design through analysis of action traces. *IEEE Transactions on Computational Intelligence and AI in Games, 7*(3), 215–228. *